TaskFlow

Software Requirements Specification

(SRS Document)

Task & Project Management Tool

Full-Stack Application — React + Spring Boot

Field	Value
Version	1.4
Date	October 12, 2026
Status	Final
Supersedes	Version 1.3 (October 12, 2026)
Classification	Confidential — For Evaluation Purposes

Revision History

Version	Date	Author	Description
0.1	Week 1	Development Team	Initial draft — scope and feature list
0.5	Week 3	Development Team	Added functional requirements and data model
0.8	Week 5	Development Team	Added non-functional requirements and API spec
1.0	Oct 12, 2026	Development Team	Final release — complete SRS
1.1	Oct 12, 2026	Development Team	Resolved ambiguities: definition of Done, single-Owner rule, DB-backed refresh tokens, optimistic locking, soft-delete rules, gap-based positioning, comment permissions, column deletion rule. Redis made optional. Deferred items listed explicitly. See Appendix A.1.
1.2	Oct 12, 2026	Development Team	Final. Added Scope Priority Split (1.5) and Architectural Decisions Log (Appendix B). Aligned priorities across 1.5, 2.2 and requirement tables; unified timeline and demo-reset path; fixed section numbering; added refresh-queue note to FR-AUTH-03; clarified soft-delete handling for native queries. See Appendix A.2.
1.3	Oct 12, 2026	Development Team	Added Section 10: Future Enhancements (Version 2) — AI integration roadmap. Defines AI feature scope, design rules (server-side context resolution, prompt injection mitigation, suggestion-only writes), tier mapping to subscription plans, and deferred classification. See Appendix A.3.
1.4	Oct 12, 2026	Development Team	Database changed from PostgreSQL to MySQL 8.0+ (InnoDB, utf8mb4). Column entity stored in table board_columns; timestamps as DATETIME(6) UTC; Flyway MySQL module; row-lock serialization of card moves (NFR-REL-06, AD-13); Testcontainers MySQL. Appendix B renumbered to AD-01..AD-14. See Appendix A.4.

Section 8 (Version 2 payment roadmap) and Section 10 (Version 2 AI enhancements) are both deferred to Version 2.0 and are unchanged from their respective prior versions except for editorial consistency.

Table of Contents

1. Introduction **5**

1.1 Purpose . . . 5

1.2 Document Conventions . . . 5

1.3 Intended Audience . . . 5

1.4 Product Scope . . . 5

1.5 Scope Priority Split (Must / Should / Defer) . . . 6

1.6 References . . . 6

2. Overall Description **8**

2.1 Product Perspective . . . 8

2.2 Product Functions Summary . . . 8

2.3 User Classes and Characteristics . . . 8

2.4 Operating Environment . . . 9

2.5 Design and Implementation Constraints . . . 9

2.6 Assumptions and Dependencies . . . 9

3. Specific Requirements — Functional **10**

3.1 Authentication & Authorization (FR-AUTH) . . . 10

3.2 Board Management (FR-BOARD) . . . 10

3.3 Card/Task Management (FR-CARD) . . . 11

3.4 Real-Time Collaboration (FR-RT) . . . 12

3.5 Activity Log (FR-ACT) . . . 12

3.6 Dashboard & Analytics (FR-DASH) . . . 12

3.7 User Profile & Settings (FR-PROF) . . . 13

3.8 Demo Mode (FR-DEMO) . . . 13

4. External Interface Requirements **14**

4.1 User Interfaces . . . 14

4.2 Hardware Interfaces . . . 14

4.3 Software Interfaces . . . 14

4.4 Communication Interfaces . . . 14

5. Non-Functional Requirements **14**

5.1 Performance . . . 14

5.2 Security . . . 15

5.3 Reliability & Availability . . . 15

5.4 Scalability . . . 15

5.5 Maintainability . . . 15

5.6 Portability . . . 16

6. System Models **16**

6.1 Data Model (ER Description) . . . 16

6.2 Key Use Cases . . . 17

6.3 API Endpoint Summary . . . 17

7. Technology Stack **19**

8. Version 2 Roadmap — Payment Gateway **19**

8.1 Subscription Tiers . . . 19

8.2 Payment Functional Requirements (Version 2) . . . 20

8.3 Payment Architecture (Version 2) . . . 20

8.4 New Entities and Endpoints (Version 2) . . . 20

9. Glossary **21**

10. Future Enhancements (Version 2) — AI Integration **22**

10.1 AI Feature Scope . . . 22

10.2 AI Design Rules . . . 22

10.3 Subscription Tier Mapping . . . 23

10.4 AI API Endpoints (Version 2) . . . 23

Appendix A: Change Log **24**

A.1 Version 1.0 to 1.1 . . . 24

A.2 Version 1.1 to 1.2 . . . 24

A.3 Version 1.2 to 1.3 . . . 25

A.4 Version 1.3 to 1.4 . . . 25

Appendix B: Architectural Decisions Log **26**

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for TaskFlow, a web-based task and project management application. TaskFlow enables individuals and teams to organize work using Kanban-style boards, manage tasks with drag-and-drop interactions, collaborate in real time, and track progress through an analytics dashboard.

This document is the primary reference for development, testing, and stakeholder review. It covers Version 1.4 of the specification for product Version 1.0. Payment gateway integration and subscription management are deferred to Version 2.0 (Section 8). AI-powered features are also deferred to Version 2.0 (Section 10).

1.2 Document Conventions

- **FR-XXX** — Functional Requirement identifier (e.g., FR-AUTH-01)
- **NFR-XXX** — Non-Functional Requirement identifier
- **[Version 2]** — Feature deferred to Version 2.0 (payment gateway, AI integration)
- **[Deferred]** — Feature kept in the specification but scheduled after the first deployed release of Version 1.0 (see 1.4)
- **Priority:** Must Have (M) | Should Have (S) | Nice to Have (N)
- Rows shaded green in requirement tables are new or changed since Version 1.0.
- Terms in the Glossary (Section 9) have the meaning defined there.

1.3 Intended Audience

- Development team — backend and frontend engineers implementing the system
- QA/Testers — writing test cases from functional requirements
- Project stakeholders / Evaluators — reviewing scope and feasibility
- Future maintainers — understanding design decisions and architecture

1.4 Product Scope

TaskFlow is a full-stack web application built with React (frontend) and Spring Boot (backend). It provides a Trello/Jira-inspired experience for small teams and individual users. The system supports:

- User registration and JWT-based authentication (access + refresh tokens) with role-based access control
- Creation and management of Kanban boards with customizable columns
- Task cards with labels, due dates, multiple assignees, descriptions, and comments
- Drag-and-drop card movement across columns with real-time synchronization
- WebSocket-based live updates so all connected users see changes instantly
- Activity logging for every action performed on a board
- Analytics dashboard with completion rates, overdue tracking, and member workload
- Demo mode with seeded data for portfolio presentation

Deferred until after the first deployed release of Version 1.0

The following are specified but are not required for the first deployed release. They are marked **[Deferred]** where they appear:

- Card file attachments (FR-CARD-10)
- Avatar image upload (part of FR-PROF-01; initials are shown instead)
- Account deletion (FR-PROF-04)
- Email verification and password-reset emails (Section 4.3; mocked in development)
- Presence / typing indicator (FR-RT-09)
- Redis-based query caching (Redis is optional in V1)

Out of Scope for Version 1.0

- Payment gateway / subscription billing (planned for Version 2.0 — Section 8)
- AI-powered features (planned for Version 2.0 — Section 10)
- Mobile native applications (responsive web only)
- File storage beyond basic attachments (no CDN/blob storage)
- Third-party integrations (Slack, GitHub, etc.)
- Advanced reporting or export to PDF/Excel
- Ownership transfer between users

1.5 Scope Priority Split (Must / Should / Defer)

The requirements are grouped into three delivery tiers to fit an 8-10 week timeline. Priorities here match the Pri column of every requirement table and Section 2.2.

Must Have (build all — Weeks 1-6)

- Auth with JWT access + refresh tokens (refresh stored in a DB table, not Redis)
- Axios interceptor with automatic retry after token refresh (concurrent 401s share a single refresh call)
- Board CRUD with per-board roles (Owner, Editor, Viewer)
- Column CRUD with is_done flag and reorder support
- Card CRUD with labels (up to 6, predefined palette), multiple assignees (join table), due dates, and the overdue indicator
- Drag-and-drop with gap-based BIGINT positions (server computes midpoint)
- Optimistic locking (@Version) on cards for conflict handling
- Soft deletes (deleted_at) on boards and cards — all queries include the filter
- WebSocket real-time updates (STOMP over WebSocket, JWT-authenticated)
- Activity log (append-only, immutable)
- Basic cross-board dashboard: overdue count and board summary
- Profile: name edit and password change
- Seeded demo data (3 users, sample board with 10-15 cards via Flyway) and Demo Mode banner

Should Have (build if on schedule — Weeks 6-8)

- Comments on cards (Viewers may comment — resolves the FR-CARD-11 vs. Viewer read-only conflict)
- Dashboard charts: task completion per week, member workload distribution
- Cards due in the next 7 days widget
- Recent activity feed across all the user's boards
- Demo bot (moves a random card every 60 s, gated by DEMO_MODE) and demo reset endpoint (/api/v1/demo/reset)

Defer to After Deployment (post-V1 release)

- File attachments on cards (requires a storage strategy — local/S3)
- Avatar upload (requires file handling)
- Account deletion (requires cascade logic review)
- Email verification (requires SMTP setup)
- Presence indicator (user is viewing/typing)
- Redis query cache (performance optimization, not needed for V1 launch)
- Board-level labels (V1 uses card-level labels with a predefined palette)

1.6 References

Ref	Document	Description
[1]	IEEE 830-1998	Recommended Practice for Software Requirements Specifications

Ref	Document	Description
[2]	Spring Boot 3 Documentation	https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/
[3]	React 18 Documentation	https://react.dev/
[4]	RFC 7519	JSON Web Token (JWT) Specification
[5]	RFC 6455	The WebSocket Protocol

2. Overall Description

2.1 Product Perspective

TaskFlow is a new, self-contained product that follows a client-server architecture:

- **Frontend (Client):** Single-page application built with React 18, TypeScript, Vite, and TailwindCSS. Communicates with the backend via REST APIs and a WebSocket connection.
- **Backend (Server):** Spring Boot 3 application exposing RESTful APIs, handling authentication, business logic, and real-time event broadcasting. Uses Spring Security with JWT, Spring Data JPA, and Spring WebSocket (STOMP).
- **Database:** MySQL 8.0+ (InnoDB, utf8mb4) for all persistent data, including refresh tokens. Flyway manages schema migrations and seed data.
- **Cache (optional):** Redis may be added for query caching. No V1 requirement depends on Redis.

The system is deployed as Docker containers (backend + database) with the frontend served as static assets from a CDN or reverse proxy.

2.2 Product Functions Summary

Module	Key Functions	Priority
Authentication & Auth	Register, login, JWT refresh with rotation, role-based access	Must Have
Board Management	Create, read, update, soft-delete boards; manage columns	Must Have
Card/Task Management	Create, edit, move, soft-delete cards; labels, due dates, multiple assignees, optimistic locking	Must Have
Drag & Drop	Reorder cards, move between columns with live sync (gap-based positions)	Must Have
Real-Time Updates	WebSocket push of committed board changes to all connected clients	Must Have
Activity Log	Track all actions with timestamps and user attribution (append-only)	Must Have
Dashboard (basic)	Cross-board overdue count and board summary	Must Have
Dashboard (charts)	Completion per week, member workload, due-soon list, recent activity	Should Have
Comments	Card-level comments by any board member, including Viewers	Should Have
User Profile	View/edit name, change password	Must Have
Demo Mode (core)	Seeded demo data and demo banner	Must Have
Demo Mode (extras)	Demo bot and reset endpoint	Should Have
[Deferred] Attachments, avatar, account deletion, email verification, presence	See Section 1.4	Deferred
[Version 2] Payments	Stripe subscriptions, plan tiers, webhook handling	Deferred
[Version 2] AI	AI-powered descriptions, task breakdown, activity summaries	Deferred

2.3 User Classes and Characteristics

User Class	Description	Permissions	Frequency
Owner	Board creator. Exactly one Owner per board. Full control over board settings, members, and content.	All actions including manage members and delete board.	Daily
Editor	Team member who can create, edit, and move cards.	Create/edit/move/delete cards, comment. Cannot manage members.	Daily
Viewer	Read-only member for board content. Receives live updates.	View cards and activity log, and post comments. Cannot create, edit, move, or delete cards or columns.	As needed

User Class	Description	Permissions	Frequency
Admin	System administrator (system-level role, separate from board roles).	User management, system settings, demo reset.	Weekly
Unauthenticated	Not logged in. Can only see the landing/login page.	None.	Once

2.4 Operating Environment

Client-side

- Modern web browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Minimum screen resolution: 1280 x 720
- JavaScript enabled; WebSocket support required for real-time features

Server-side

- Java 17+ (LTS) and Spring Boot 3.x
- MySQL 8.0+ (InnoDB, utf8mb4)
- Redis 6+ (optional)
- Docker 20+ and Docker Compose
- Linux/Unix-based hosting environment

2.5 Design and Implementation Constraints

- Language: backend in Java 17+, frontend in TypeScript/React
- Framework: Spring Boot 3 (backend), Vite + React 18 (frontend)
- Database: MySQL only — no NoSQL
- Auth: JWT-based — no server-side sessions for API calls; refresh tokens are persisted (hashed) in MySQL
- API style: RESTful JSON APIs — no GraphQL
- Real-time: WebSocket over STOMP — no Server-Sent Events or polling
- Migrations: Flyway — no manual SQL execution
- Deletion: boards and cards use soft delete; every query must exclude soft-deleted rows
- Timeline: 8-10 weeks (about 2 months) for Version 1.0

2.6 Assumptions and Dependencies

- Users have a modern web browser with internet connectivity
- The deployment environment supports Docker containers
- MySQL 8.0+ is available as a managed service or Docker container
- No legacy system integration is required
- All user data is stored in the relational database — no external identity provider (OAuth) in V1
- [Deferred] An email service is available for account verification (mocked in development)

3. Specific Requirements — Functional

3.1 Authentication & Authorization (FR-AUTH)

The system shall provide secure user authentication using JSON Web Tokens (JWT) and enforce role-based access control (RBAC) at both the API and UI levels.

ID	Requirement	Pri	Notes
FR-AUTH-01	The system shall allow users to register with email, name, and password.	M	Password min 8 chars, hashed with BCrypt.
FR-AUTH-02	The system shall authenticate users and return a JWT access token and a refresh token.	M	Access token TTL: 15 min. Refresh token TTL: 7 days.
FR-AUTH-03	The system shall provide a refresh endpoint to obtain new access tokens. Refresh tokens shall be stored only as hashes in the database and rotated on every use. Presenting a revoked or expired refresh token shall be rejected.	M	Table refresh_tokens. Client retries the failed request once after a successful refresh. Concurrent 401s must share one in-flight refresh call (queue them), because rotation invalidates the old token.
FR-AUTH-04	The system shall support logout by revoking the refresh token (setting revoked_at).	M	Database-backed; Redis is not required.
FR-AUTH-05	The system shall enforce three board-level roles: Owner, Editor, Viewer.	M	Checked on every board-scoped API call in the service layer.
FR-AUTH-06	The system shall allow the Owner to invite members by email.	M	Invited user must have an existing account or register.
FR-AUTH-07	The system shall allow the Owner to remove Editors and Viewers from a board. Each board has exactly one Owner, who cannot be removed or demoted.	M	Removing a member unassigns them from all cards on that board.
FR-AUTH-08	The system shall allow the Owner to change a non-Owner member's role between Editor and Viewer.	M	Ownership transfer is out of scope for V1.
FR-AUTH-09	The system shall restrict Viewer users from creating, editing, moving, or deleting cards and columns.	M	Frontend (UI disabled) and backend (403). Viewers may comment (FR-CARD-11).
FR-AUTH-10	The system shall support a system-level Admin role for user management.	S	Admin can view all users and reset demo data.
FR-AUTH-11	The system shall validate the JWT on every WebSocket connection.	M	STOMP handshake interceptor.
FR-AUTH-12	The system shall respond 404 (not 403) when a non-member requests a board, card, or column of that board.	M	Prevents leaking which resource ids exist.

3.2 Board Management (FR-BOARD)

A Board is the top-level organizational unit. Each board contains an ordered list of Columns, which contain Cards. Users can belong to multiple boards with different roles.

ID	Requirement	Pri	Notes
FR-BOARD-01	The system shall allow authenticated users to create a board with a name and description.	M	Creator becomes the Owner.
FR-BOARD-02	The system shall display all boards the user is a member of on the dashboard.	M	Sorted by last activity.
FR-BOARD-03	The system shall allow the Owner to edit board name and description.	M	
FR-BOARD-04	The system shall allow the Owner to delete a board and all associated data.	M	Soft delete (deleted_at) with confirmation. All queries exclude soft-deleted rows.
FR-BOARD-05	The system shall initialize new boards with default columns: To Do, In Progress, Done.	M	The Done column is created with is_done = true. Customizable afterwards.

ID	Requirement	Pri	Notes
FR-BOARD-06	The system shall allow Owner/Editor to add, rename, reorder, and delete columns. A column that contains cards cannot be deleted until it is empty.	M	Column order is persisted. Deleting a non-empty column returns 409.
FR-BOARD-07	The system shall enforce a maximum of 50 boards per user.	S	Prevents abuse.
FR-BOARD-08	The system shall enforce a maximum of 20 columns per board.	S	
FR-BOARD-09	The system shall display the board's last activity timestamp.	S	
FR-BOARD-10	The system shall allow board members to search cards by title within a board.	S	Client-side filter for V1.
FR-BOARD-11	The Owner shall be able to mark or unmark any column as a Done column. A board may have more than one.	M	Sets board_columns.is_done. Existing cards in the column are not back-filled with completed_at.

3.3 Card/Task Management (FR-CARD)

Cards are the fundamental work unit. Each card belongs to a column and can be moved between columns via drag-and-drop or API calls. A card is considered **Done** when its column has is_done = true.

ID	Requirement	Pri	Notes
FR-CARD-01	The system shall allow Owner/Editor to create a card in any column with a title.	M	Title required. Description optional.
FR-CARD-02	The system shall allow editing card title, description, due date, and labels.	M	
FR-CARD-03	The system shall allow assigning one or more board members to a card.	M	Multi-assign supported. Assignees must be members of the card's board.
FR-CARD-04	The system shall support color-coded labels (up to 6 per card).	M	Card-level labels from a predefined palette of 6 colors.
FR-CARD-05	The system shall allow drag-and-drop reordering of cards within a column.	M	Positions are BIGINT with gaps of 1000. Server computes the new position as the midpoint of the neighbouring cards.
FR-CARD-06	The system shall allow drag-and-drop movement of cards between columns.	M	Request carries targetColumnId, beforeCardId, afterCardId. Target column must belong to the same board. If the gap between neighbours falls below 2, the column is renumbered in the same transaction. Moves within one column are serialized with a row lock on that column.
FR-CARD-07	The system shall allow Owner/Editor to delete a card.	M	Soft delete (deleted_at). Excluded from all queries and counts.
FR-CARD-08	The system shall display overdue cards with a visual indicator.	M	Overdue = due_date before today AND the card's column has is_done = false AND card is not deleted.
FR-CARD-09	The system shall display the assignee's initials on each card.	S	Avatar images are deferred.
FR-CARD-10	[Deferred] The system shall allow attaching a single file (image or PDF, max 5 MB) to a card.	S	Local filesystem in V1.
FR-CARD-11	The system shall support card-level comments. Any board member, including Viewers, can comment.	S	Comment authors can delete their own comments; Owner can delete any.
FR-CARD-12	The system shall enforce a maximum of 200 non-deleted cards per board.	S	
FR-CARD-13	Concurrent edits to the same card shall be detected using optimistic locking. A stale update shall return 409 and the client shall refetch the card.	M	cards.version (JPA @Version). Clients converge on the broadcast state.

ID	Requirement	Pri	Notes
FR-CARD-14	When a card enters a Done column, the system shall set completed_at. When it leaves a Done column, completed_at shall be cleared.	M	Used by FR-DASH-03.

3.4 Real-Time Collaboration (FR-RT)

The system shall use WebSocket (STOMP over WebSocket) to push real-time updates to all clients currently viewing the same board.

ID	Requirement	Pri	Notes
FR-RT-01	The system shall broadcast card creation events to all connected board members.	M	
FR-RT-02	The system shall broadcast card move/reorder events in real time.	M	Include new column_id and position.
FR-RT-03	The system shall broadcast card update events (title, description, labels, due date, assignees).	M	
FR-RT-04	The system shall broadcast card deletion events.	M	
FR-RT-05	The system shall broadcast column add/rename/reorder/delete events.	M	
FR-RT-06	The system shall authenticate WebSocket connections using JWT.	M	STOMP handshake interceptor.
FR-RT-07	The system shall scope WebSocket subscriptions to board-level channels and reject subscriptions from non-members.	M	Topic: /topic/board/{boardId}. SUBSCRIBE frames are checked against board membership.
FR-RT-08	The system shall handle client disconnection gracefully and resume on reconnect.	S	Client reconnects with backoff, re-subscribes, and refetches the full board.
FR-RT-09	[Deferred] The system shall show a 'user is typing/viewing' indicator.	N	Presence channel.
FR-RT-10	Events shall be broadcast only after the database transaction has committed.	M	Prevents clients seeing rolled-back data. Each event includes the sender's user id so clients can ignore their own echo.

3.5 Activity Log (FR-ACT)

Every significant action on a board shall be recorded in an immutable activity log, attributed to the acting user with a timestamp.

ID	Requirement	Pri	Notes
FR-ACT-01	The system shall log card creation, updates, moves, and deletions.	M	
FR-ACT-02	The system shall log column changes (add, rename, reorder, delete).	M	
FR-ACT-03	The system shall log member additions, removals, and role changes.	M	
FR-ACT-04	The system shall display the activity log in a sidebar or tab on the board view.	M	Paginated, newest first.
FR-ACT-05	The system shall include user name, action description, and relative timestamp.	M	e.g., 'Bob moved Card X to Done — 2 min ago'.
FR-ACT-06	The system shall make activity logs immutable (no edit/delete).	M	Append-only table.

3.6 Dashboard & Analytics (FR-DASH)

The dashboard provides a cross-board overview of the user's tasks and project health, covering all boards the user belongs to.

ID	Requirement	Pri	Notes
FR-DASH-01	The system shall display a summary of the user's boards with card counts per column.	M	
FR-DASH-02	The system shall show the count of overdue cards across all the user's boards.	M	Overdue as defined in FR-CARD-08.
FR-DASH-03	The system shall display a task completion chart (cards completed per week).	S	Based on cards.completed_at. Bar or line chart.
FR-DASH-04	The system shall show a member workload chart (cards assigned per member).	S	Counts non-deleted, non-Done cards via card_assignees.
FR-DASH-05	The system shall display recent activity across all the user's boards.	S	Last 20 activities.
FR-DASH-06	The system shall show cards due in the next 7 days.	S	Sorted by due date; excludes Done cards.

3.7 User Profile & Settings (FR-PROF)

ID	Requirement	Pri	Notes
FR-PROF-01	The system shall allow users to view and edit their name.	M	Avatar upload is [Deferred]; initials are displayed.
FR-PROF-02	The system shall allow users to change their password.	M	Requires current password. Revokes existing refresh tokens.
FR-PROF-03	The system shall display the user's boards and role on each.	M	
FR-PROF-04	[Deferred] The system shall allow users to delete their account.	S	Soft delete via users.deleted_at. Owned boards must be transferred or deleted first.

3.8 Demo Mode (FR-DEMO)

Demo mode provides pre-seeded data and a demo bot for portfolio presentation. It is activated by the environment variable DEMO_MODE=true.

ID	Requirement	Pri	Notes
FR-DEMO-01	The system shall seed 3 demo users: Alice (Owner), Bob (Editor), Carol (Viewer).	M	Via Flyway migration, demo profile only.
FR-DEMO-02	The system shall seed a sample board 'Job Search 2026' with 4 columns and 10-15 cards.	M	Includes overdue, upcoming, labels, assignees; one Done column.
FR-DEMO-03	The system shall display a 'Demo Mode' banner in the UI when active.	M	
FR-DEMO-04	The system shall provide a demo bot that moves a random card every 60 seconds.	S	Gated by DEMO_MODE.
FR-DEMO-05	The system shall provide a /api/v1/demo/reset endpoint to restore seed data.	S	Admin-only or env-gated.
FR-DEMO-06	The system shall seed activity log history for the sample board.	S	Makes the activity log look populated.

4. External Interface Requirements

4.1 User Interfaces

Screen	Description	Key Elements
Landing / Login	Public page with login and registration forms.	Email/password fields, 'Try Demo' button.
Dashboard	Main hub after login. Boards and analytics.	Board grid, overdue count, completion chart, recent activity.
Board View	Kanban board with columns and cards.	Column headers, cards, drag-and-drop zones, member initials. Drag handles and edit controls hidden for Viewers.
Card Detail Modal	Overlay with full card details.	Title, description, labels, due date, assignees, comments, activity.
Board Settings	Owner-only configuration page.	Name/description edit, member list with roles, invite form, Done-column toggle, delete board.
User Profile	Account settings page.	Name, password change, board list with roles.
Admin Panel	System admin page (Admin only).	User list, demo reset button, system stats.

4.2 Hardware Interfaces

No specific hardware interfaces are required. Minimum server specification: 2 vCPU, 4 GB RAM, 20 GB storage.

4.3 Software Interfaces

Interface	Technology	Purpose
Database	MySQL 8.0+ (InnoDB)	Primary data store, including hashed refresh tokens
Cache	Redis 6+ (optional)	Optional query cache. Not required for token handling.
Email	[Deferred] SMTP (Spring Mail)	Account verification, password reset (mocked in dev)
WebSocket	STOMP over WebSocket	Real-time event broadcasting
File Storage	[Deferred] Local filesystem	Card attachments; S3-compatible in V2

4.4 Communication Interfaces

Protocol	Endpoint Pattern	Purpose
HTTPS (REST)	/api/v1/**	All CRUD operations, authentication
WSS (STOMP)	/ws	Real-time board updates
HTTPS	/api/v1/demo/**	Demo mode endpoints

5. Non-Functional Requirements

5.1 Performance

ID	Requirement	Target
NFR-PERF-01	API response time for standard CRUD operations.	< 200 ms at p95 under normal load.
NFR-PERF-02	WebSocket event delivery latency.	< 500 ms from action to client receipt.
NFR-PERF-03	Initial page load time (first meaningful paint).	< 2 seconds on 10 Mbps connection.
NFR-PERF-04	Board rendering with maximum cards (200).	< 1 second to fully interactive.
NFR-PERF-05	Concurrent users per board.	At least 20 simultaneous connections.
NFR-PERF-06	Database query time for dashboard aggregations.	< 300 ms.
NFR-PERF-07	Full-board load shall not issue N+1 queries.	Fixed number of queries regardless of card count.

5.2 Security

ID	Requirement
NFR-SEC-01	Passwords shall be hashed using BCrypt with strength 10 or higher.
NFR-SEC-02	JWT access tokens shall expire within 15 minutes.
NFR-SEC-03	All API endpoints shall be protected except /api/v1/auth/**.
NFR-SEC-04	CORS shall be configured to allow only the frontend origin.
NFR-SEC-05	Input validation shall be performed on all API inputs (server-side).
NFR-SEC-06	SQL injection shall be prevented via JPA parameterized queries.
NFR-SEC-07	XSS shall be prevented via React's default escaping and CSP headers.
NFR-SEC-08	CSRF protection is disabled for the stateless JWT API, because tokens are sent in the Authorization header and not in cookies. If cookies are ever used for tokens, CSRF protection shall be re-enabled.
NFR-SEC-09	Sensitive data (passwords, tokens) shall not appear in logs.
NFR-SEC-10	HTTPS shall be enforced in production.
NFR-SEC-11	Refresh tokens shall be stored only as hashes; the raw token is never persisted.
NFR-SEC-12	Authentication endpoints shall be rate limited to mitigate brute-force attacks.

5.3 Reliability & Availability

ID	Requirement	Target
NFR-REL-01	System uptime.	99.5% during business hours.
NFR-REL-02	Data backup frequency.	Daily automated backups.
NFR-REL-03	Graceful degradation on WebSocket failure.	App continues to work; client reconnects and refetches latest state.
NFR-REL-04	Database connection pool management.	HikariCP with sensible defaults; no connection leaks.
NFR-REL-05	Multi-step writes (card move with rebalance, member removal with unassignment) shall be atomic.	Single database transaction.
NFR-REL-06	Concurrent card moves within the same column shall be serialized, and deadlock failures retried a bounded number of times.	No lost, duplicated or out-of-order positions.

5.4 Scalability

- The backend shall be stateless (JWT-based) to allow horizontal scaling.
- The schema shall support up to 10,000 users and 50,000 boards without restructuring.
- WebSocket connections shall be scoped per board.
- NFR-SCA-01: The system shall handle 100 concurrent WebSocket connections on a single instance.

5.5 Maintainability

- Standard Java/React conventions with consistent formatting.
- Layered architecture: Controller > Service > Repository. Board-level authorization is enforced in the service layer.
- All entities have corresponding DTOs — no entity exposure in API responses.
- Test coverage target: 70% for backend services, 60% for frontend components.
- A permission test matrix (Owner / Editor / Viewer / non-member against every endpoint) and a concurrency test for card moves and stale edits shall be maintained.
- API documented with SpringDoc OpenAPI (Swagger UI).

- Database migrations managed exclusively through Flyway.

5.6 Portability

- Runs on any OS supporting Java 17+ and Node.js 18+.
- Docker Compose provided for one-command local setup.
- No OS-specific file paths — all paths configurable via application properties.

6. System Models

6.1 Data Model (ER Description)

The database consists of the following core entities. Green rows are new or changed since Version 1.0.

Entity	Key Fields	Relationships
User	id, email (unique), name, password_hash, role (USER/ADMIN), created_at, deleted_at (used by deferred FR-PROF-04)	Has many BoardMembers.
RefreshToken	id, user_id (FK), token_hash, expires_at, revoked_at, created_at	Belongs to User. Rotated on use.
Board	id, name, description, owner_id (FK > User), created_at, updated_at, deleted_at	Has many Columns, BoardMembers, ActivityLogs.
BoardMember	id, board_id (FK), user_id (FK), role (OWNER/EDITOR/VIEWER), joined_at. Unique (board_id, user_id).	Links User to Board. Exactly one OWNER per board.
Column (table: board_columns)	id, board_id (FK), name, position (BIGINT), is_done (boolean, default false), created_at	Has many Cards. Belongs to Board.
Card	id, column_id (FK), title, description, position (BIGINT, gaps of 1000), due_date, completed_at, version (optimistic lock), created_by (FK), created_at, updated_at, deleted_at	Belongs to Column. Has Labels, Assignees, Comments.
CardLabel	id, card_id (FK), name, color (palette of 6). Max 6 per card.	Belongs to Card.
CardAssignee	id, card_id (FK), user_id (FK). Unique (card_id, user_id).	Links Card to User.
Comment	id, card_id (FK), user_id (FK), content, created_at	Belongs to Card. Authored by User.
ActivityLog	id, board_id (FK), user_id (FK), action_type, entity_type, entity_id, description, created_at	Belongs to Board. Append-only.
Attachment [Deferred]	id, card_id (FK), file_name, file_path, file_size, uploaded_by (FK), created_at	Belongs to Card.

Key relationships

- User 1:N BoardMember N:1 Board (many-to-many through BoardMember with role)
- Board 1:N Column 1:N Card (hierarchical)
- Card N:N User through CardAssignee
- Card 1:N CardLabel, Comment (and Attachment when implemented)
- ActivityLog is append-only, linked to Board and User

Data conventions

- **Done:** a card is Done when its column has is_done = true. Overdue, workload and completion metrics all use this definition.
- **Soft delete:** boards, cards (and users, when FR-PROF-04 is built) are soft-deleted. Every query, count and aggregate must exclude rows with deleted_at set.
- **Positions:** columns and cards use BIGINT positions with gaps of 1000. A new item is placed at max + 1000; a moved item takes the midpoint between neighbours; a column is renumbered when the gap falls below 2.
- **Optimistic locking:** cards.version is incremented on every update; a mismatch yields HTTP 409.
- **Indexes:** cards (column_id, position), board_members (user_id), activity_log (board_id, created_at), refresh_tokens (token_hash).

- **Database engine:** MySQL 8.0+, InnoDB, utf8mb4 (utf8mb4_0900_ai_ci). The Column entity is stored in the table board_columns, because COLUMN is a reserved word in MySQL.
- **Types:** timestamps are DATETIME(6) in UTC (avoids the TIMESTAMP 2038 limit); booleans are BOOLEAN (TINYINT(1)); refresh_tokens.token_hash is CHAR(64) (SHA-256 hex) with a unique index; emails are lower-cased before storage.
- **Isolation and locking:** the InnoDB default REPEATABLE READ is kept. A card move locks the target column row (SELECT ... FOR UPDATE) so concurrent moves are serialized; deadlock failures are retried a bounded number of times.

6.2 Key Use Cases

Use Case	Actor	Precondition	Main Flow	Postcondition
UC-01 Register	Unauthenticated	Email not in system	Enter email, name, password. System validates, hashes password, creates user, returns tokens.	User is authenticated.
UC-02 Login	Unauthenticated	Registered account	Enter email + password. System validates and returns access + refresh tokens.	User is authenticated.
UC-03 Create Board	Authenticated user	Logged in	Enter name + description. System creates board, adds creator as Owner, creates default columns (Done column flagged).	Board appears on dashboard.
UC-04 Invite Member	Owner	User is Owner	Enter member email + role (Editor/Viewer). System adds BoardMember.	Member can access board.
UC-05 Create Card	Owner / Editor	On board view	Enter title in column. System creates card at max position + 1000.	Card appears in column.
UC-06 Move Card	Owner / Editor	Card exists	Drag card. Client sends target column and neighbours. System computes position, updates in one transaction, sets/clears completed_at, commits, then broadcasts.	All clients update.
UC-07 View Board (Viewer)	Viewer	Viewer on board	Open board. System returns board with role; drag handles and edit controls hidden.	Read-only view; may comment.
UC-08 View Dashboard	Authenticated user	>= 1 board	System aggregates cross-board stats, overdue counts, completion data.	Dashboard displays.
UC-09 Refresh Session	Authenticated user	Access token expired	Client sends refresh token. System verifies hash, revokes it, issues new pair. Client retries original request.	Session continues.
UC-10 Concurrent Edit	Two Editors	Same card open	Both edit. First commit succeeds; second gets 409, refetches the card and sees the updated state.	No lost update.

6.3 API Endpoint Summary

All endpoints are prefixed with /api/v1. Request/response format is JSON.

Method	Endpoint	Description	Auth
POST	/auth/register	Register new user	None
POST	/auth/login	Authenticate and get tokens	None
POST	/auth/refresh	Rotate refresh token, get new access token	Refresh token
POST	/auth/logout	Revoke refresh token	Access token
GET	/users/me	Get current user profile	Access token
PUT	/users/me	Update profile (name)	Access token
PUT	/users/me/password	Change password	Access token
GET	/boards	List user's boards	Access token
POST	/boards	Create board	Access token
GET	/boards/{id}	Get board with columns, cards, labels, assignees, caller's role	Board member
PUT	/boards/{id}	Update board	Owner only

Method	Endpoint	Description	Auth
DELETE	/boards/{id}	Soft-delete board	Owner only
POST	/boards/{id}/members	Invite member	Owner only
PUT	/boards/{id}/members/{uid}	Change member role (Editor/Viewer)	Owner only
DELETE	/boards/{id}/members/{uid}	Remove member	Owner only
POST	/boards/{id}/columns	Add column	Owner / Editor
PUT	/boards/{id}/columns/{cid}	Update column (name, is_done*)	Owner / Editor (*is_done: Owner)
PUT	/boards/{id}/columns/reorder	Reorder columns	Owner / Editor
DELETE	/boards/{id}/columns/{cid}	Delete empty column (409 if it has cards)	Owner / Editor
POST	/columns/{cid}/cards	Create card	Owner / Editor
GET	/cards/{id}	Get card detail	Board member
PUT	/cards/{id}	Update card (requires version; 409 if stale)	Owner / Editor
PUT	/cards/{id}/move	Move card: { targetColumnId, beforeCardId, afterCardId }	Owner / Editor
DELETE	/cards/{id}	Soft-delete card	Owner / Editor
GET	/cards/{id}/comments	List comments	Board member
POST	/cards/{id}/comments	Add comment	Board member (incl. Viewer)
GET	/boards/{id}/activity	Get activity log (paginated)	Board member
GET	/dashboard/stats	Cross-board dashboard summary	Access token
POST	/demo/reset	Reset demo data	Admin / DEMO_MODE

Standard error codes: 400 validation, 401 unauthenticated/expired token, 403 role not permitted, 404 not found or not a member, 409 stale version or non-empty column, 429 rate limited.

WebSocket endpoint: /ws — clients subscribe to /topic/board/{boardId} for real-time events. Events are broadcast after commit when cards are created, moved, updated, or deleted, and when columns or members change.

7. Technology Stack

Layer	Technology	Version	Purpose
Frontend Framework	React	18.x	Component-based UI
Frontend Language	TypeScript	5.x	Type safety
Build Tool	Vite	5.x	Dev server and bundling
Styling	TailwindCSS	3.x	Utility-first CSS
State Management	React Query (TanStack Query)	5.x	Server state, caching, optimistic mutations; WebSocket events update the query cache
Drag & Drop	@dnd-kit/core	6.x	Accessible drag-and-drop
Charts	Recharts	2.x	Dashboard charts
HTTP Client	Axios	1.x	REST calls; interceptor handles token refresh and retry
WebSocket Client	@stomp/stompjs (+ sockjs optional)	7.x / 1.x	STOMP over WebSocket
Backend Framework	Spring Boot	3.x	Application framework
Security	Spring Security + JWT (jjwt)	6.x / 0.12.x	Authentication and authorization
ORM	Spring Data JPA / Hibernate	3.x	Database access; @Version for optimistic locking
Database	MySQL (InnoDB)	8.0+	Relational data store; utf8mb4; default isolation REPEATABLE READ
Migrations	Flyway	Managed by Spring Boot	Schema versioning and seed data. MySQL support requires the flyway-mysql module.
Cache (optional)	Redis (Spring Cache)	6+	Optional query cache; not required by any V1 requirement
Real-Time	Spring WebSocket (STOMP)	3.x	Live event broadcasting
API Docs	SpringDoc OpenAPI	2.x	Swagger UI
Validation	Jakarta Validation	3.x	Input validation
Testing (Backend)	JUnit 5 + Mockito + MockMvc + Testcontainers	5.x	Unit, integration, permission-matrix and concurrency tests against a Testcontainers MySQL instance
Testing (Frontend)	Vitest + React Testing Library	Latest	Component tests
Containerization	Docker + Docker Compose	20+	Local dev and deployment
CI/CD	GitHub Actions	N/A	Automated build, test, deploy

8. Version 2 Roadmap — Payment Gateway

Version 2.0 introduces subscription-based pricing powered by Stripe, enabling a freemium model. These features are explicitly excluded from Version 1.0.

8.1 Subscription Tiers

Feature	Free Tier	Pro Tier ($9/mo)	Team Tier ($19/mo)
Boards	1	Unlimited	Unlimited
Members per board	3	10	Unlimited
Cards per board	50	500	Unlimited
File attachments	No	Yes (10 MB max)	Yes (25 MB max)
Activity log retention	7 days	90 days	Unlimited

Feature	Free Tier	Pro Tier ($9/mo)	Team Tier ($19/mo)
Dashboard analytics	Basic	Advanced	Advanced + Export
Priority support	No	Email	Email + Chat

8.2 Payment Functional Requirements (Version 2)

ID	Requirement	Priority
V2-FR-PAY-01	Integrate Stripe SDK for subscription management.	M
V2-FR-PAY-02	Users shall be able to view current plan and billing info.	M
V2-FR-PAY-03	Users shall be able to upgrade/downgrade plans via Stripe Checkout.	M
V2-FR-PAY-04	The system shall handle Stripe webhooks for payment success, failure, and cancellation.	M
V2-FR-PAY-05	The system shall enforce plan limits (boards, members, cards) at the API level.	M
V2-FR-PAY-06	The system shall display a paywall/upgrade prompt when limits are reached.	M
V2-FR-PAY-07	Users shall receive email notifications for payment events.	S
V2-FR-PAY-08	The system shall support coupon codes.	S
V2-FR-PAY-09	Admin shall be able to view all subscriptions and revenue.	S
V2-FR-PAY-10	The system shall support annual billing with a discount.	N

8.3 Payment Architecture (Version 2)

- **Stripe Checkout Session:** frontend redirects to Stripe-hosted checkout; Stripe redirects back to a success URL.
- **Stripe Webhooks:** backend verifies the signature and handles checkout.session.completed, customer.subscription.updated, invoice.payment_failed, and similar events idempotently.
- **Subscription entity:** links User to Stripe customer id, subscription id, plan tier, status, and current period end.
- **Plan enforcement:** an interceptor checks limits and returns 402 Payment Required with details.
- **Customer Portal:** users manage billing via the Stripe Customer Portal.

8.4 New Entities and Endpoints (Version 2)

Entity	Key Fields
Subscription	id, user_id (FK), stripe_customer_id, stripe_subscription_id, plan_tier, status, current_period_end, created_at
Plan	id, name, tier (FREE/PRO/TEAM), max_boards, max_members, max_cards, price_monthly, price_annually
PaymentEvent	id, user_id (FK), event_type, stripe_event_id (unique), amount, status, created_at

Method	Endpoint	Description
GET	/api/v2/billing/subscription	Get current subscription
POST	/api/v2/billing/checkout	Create Stripe Checkout session
POST	/api/v2/billing/portal	Create Customer Portal session
POST	/api/v2/billing/webhook	Receive Stripe webhook events
GET	/api/v2/billing/plans	List plans
POST	/api/v2/billing/coupon	Validate and apply coupon

9. Glossary

Term	Definition
Board	Top-level organizational unit containing columns and cards.
Card	A task within a column: title, description, labels, due date, assignees.
Column	A vertical lane representing a stage (e.g., To Do, In Progress, Done).
Done column	A column with is_done = true. Cards in it count as completed.
Overdue	A non-deleted card with a due date before today whose column is not a Done column.
Soft delete	Marking a row deleted with a timestamp (deleted_at) instead of removing it. Such rows are excluded from all queries.
Optimistic locking	Concurrency control using a version number; an update with a stale version is rejected (HTTP 409).
Refresh token	Long-lived token (7 days) used to obtain new access tokens. Stored only as a hash and rotated on each use.
Gap-based positioning	Ordering with spaced integers (1000, 2000, ...) so moving an item changes one row.
JWT	JSON Web Token — compact, URL-safe token carrying identity and expiry.
RBAC	Role-Based Access Control — permissions assigned to roles.
STOMP	Simple Text Oriented Messaging Protocol, used over WebSocket for message routing.
WebSocket	Full-duplex communication protocol over a single TCP connection.
DTO	Data Transfer Object decoupling API responses from database entities.
Flyway	Database migration tool applying versioned SQL scripts.
BCrypt	Password hashing function based on the Blowfish cipher.
Kanban	Visual workflow method using boards, columns, and cards.
Owner / Editor / Viewer	Board-level roles: full control / create-edit-move cards / read-only plus comments.
Demo Mode	Mode enabled by DEMO_MODE=true: seeded data and a demo bot.
Stripe	Payment platform used in Version 2.
LLM	Large Language Model — AI model used for text generation tasks (Version 2 AI features).
Prompt injection	Attack vector where malicious input in user-controlled text manipulates an LLM's behaviour. Mitigated in V2 AI features via strict templates and output validation.
InnoDB	MySQL's transactional storage engine, providing row-level locking and foreign keys.

10. Future Enhancements (Version 2) — AI Integration

Version 2.0 introduces AI-powered features to enhance productivity within TaskFlow. These features are built on top of a stable V1 deployment and are gated behind an environment flag (AI_ENABLED=true). The core application functions fully without AI. AI capabilities are tied to the subscription tier system defined in Section 8, providing a differentiator for paid plans.

All AI features follow a suggestion-only model: the AI returns draft content, and the user must explicitly accept it. The normal update endpoints then perform the write, so optimistic locking, activity logging, WebSocket broadcasts, and all existing validation still apply. AI never writes to the database directly.

10.1 AI Feature Scope

The following AI features are planned for Version 2.0, ranked by implementation priority:

ID	Feature	Description	Pri
V2-FR-AI-01	Smart Card Description Generation	Given a card title and board context (resolved server-side from cardId), generate a suggested description. Available to Pro and Team tiers.	M
V2-FR-AI-02	Task Breakdown / Subtask Suggestions	Given a card title, suggest subtasks or break a large task into smaller actionable items. Available to Pro and Team tiers.	M
V2-FR-AI-03	Natural Language Card Creation	Parse natural language input (e.g., 'Assign login bug fix to Bob, due Friday, label it Bug') into structured card fields. Available to Team tier.	S
V2-FR-AI-04	Activity Summarization	Summarize the activity log into a human-readable digest (e.g., 'What happened on this board today?'). Available to Team tier.	S
V2-FR-AI-05	Semantic Card Search	Semantic search across cards beyond title matching. Available to Team tier.	N

Excluded features: smart due date suggestions and predictive overdue alerts are excluded because they require historical completion data that does not exist in a new application or demo environment. These are analytics/ML problems, not LLM calls, and would be thin in practice. Workload balancing suggestions overlap with the existing workload chart (FR-DASH-04) and are deferred to a later release.

10.2 AI Design Rules

All AI features must comply with the following design rules to ensure security, data isolation, and system integrity:

Rule	Description	Rationale
R1: Server-side context resolution	AI endpoints accept a cardId (or boardId), not free-form context from the client. The server resolves the card/board through the existing BoardAccessService check and builds the LLM prompt from data the requesting user is authorized to see.	Prevents cross-board data leakage. A user cannot craft a request that pulls another board's content into a prompt. Viewers cannot use AI endpoints (Editor or Owner only).
R2: Prompt injection mitigation	Card titles and descriptions authored by other board members are treated as untrusted input. Prompt templates are strict and structured. Output is validated and rendered as plain text. React's default escaping provides the final XSS defence.	Any board member can put arbitrary text in a card. If that text flows into an LLM prompt, it becomes a jailbreak vector. Strict templates and output validation mitigate this.
R3: Suggestion-only writes	AI returns draft content to the client. The user must explicitly accept it. The normal update endpoint (PUT /cards/{id}) performs the write, so optimistic locking (cards.version), activity logging, and WebSocket broadcasts all still apply.	AI never writes to the database directly. All existing concurrency control, audit trails, and real-time sync remain intact.
R4: Environment-gated	All AI features are gated by AI_ENABLED=true. When disabled, AI endpoints return 501 Not Implemented. The core application functions fully without AI.	No AI dependency in the core flow. The feature can be turned off entirely if the LLM provider is unavailable or costs are too high.
R5: Server-side API key management	LLM API keys are stored server-side only (environment variables or secrets manager). They are never exposed to the client.	Prevents API key theft and unauthorized usage.

Rule	Description	Rationale
R6: Rate limiting and cost control	AI endpoints are rate-limited per user and per tier. Daily/monthly request caps are enforced at the API level.	Controls LLM API costs and prevents abuse.

10.3 Subscription Tier Mapping

AI features are mapped to the subscription tiers defined in Section 8.1:

Feature	Free Tier	Pro Tier ($9/mo)	Team Tier ($19/mo)
Smart Description Generation (V2-FR-AI-01)	Not available	50 requests/day	200 requests/day
Task Breakdown (V2-FR-AI-02)	Not available	30 requests/day	100 requests/day
Natural Language Card Creation (V2-FR-AI-03)	Not available	Not available	100 requests/day
Activity Summarization (V2-FR-AI-04)	Not available	Not available	50 requests/day
Semantic Card Search (V2-FR-AI-05)	Not available	Not available	Unlimited

10.4 AI API Endpoints (Version 2)

All AI endpoints are prefixed with /api/v2/ai and require a valid access token. When AI_ENABLED=false, all endpoints return 501 Not Implemented.

Method	Endpoint	Description	Auth
POST	/ai/suggest-description	Generate a card description suggestion. Accepts { cardId }. Server resolves card context through BoardAccessService. Returns { suggestedDescription }.	Access token (Editor/Owner only)
POST	/ai/suggest-subtasks	Generate subtask suggestions for a card. Accepts { cardId }. Server resolves card context. Returns { subtasks: string[] }.	Access token (Editor/Owner only)
POST	/ai/parse-card	Parse natural language into structured card fields. Accepts { boardId, columnId, input }. Server validates board/column access. Returns { title, description, dueDate, labels, assigneeIds }.	Access token (Editor/Owner only, Team tier)
POST	/ai/summarize-activity	Summarize board activity. Accepts { boardId, timeRange }. Server validates board access. Returns { summary: string }.	Access token (Board member, Team tier)
POST	/ai/search	Semantic search across cards. Accepts { boardId, query }. Server validates board access. Returns { cards: CardSummary[] }.	Access token (Board member, Team tier)

Standard error codes for AI endpoints: 400 validation (invalid cardId, empty input), 401 unauthenticated, 403 role not permitted or tier does not include AI, 404 card/board not found or not a member, 429 rate limited (daily/monthly cap exceeded), 501 AI disabled (AI_ENABLED=false), 503 LLM provider unavailable.

Appendix A: Change Log

A.1 Version 1.0 to 1.1

Area	Change	Reason
Definition of Done	Added columns.is_done, cards.completed_at, FR-BOARD-11, FR-CARD-14; redefined FR-CARD-08, FR-DASH-02/03/04/06.	Customizable columns made 'Done' undefined.
Owner rules	FR-AUTH-07/08 now state exactly one Owner who cannot be removed or demoted. 'Last Owner' wording removed.	boards.owner_id is a single column.
Refresh tokens	FR-AUTH-03/04 use hashed, rotated, DB-stored refresh tokens. New entity RefreshToken, NFR-SEC-11. Redis made optional.	Removes Redis dependency; enables revocation.
Comments	FR-CARD-11 allows Viewers to comment; Viewer role text in 2.3 and FR-AUTH-09 aligned.	Contradiction between 'read-only' and 'any member can comment'.
Column deletion	FR-BOARD-06: non-empty columns cannot be deleted (409).	Behaviour was unspecified.
Soft delete	deleted_at added to Board, Card, User; rule that all queries exclude deleted rows (2.5, 6.1).	Prevents leaked or miscounted rows.
Optimistic locking	New FR-CARD-13, cards.version, UC-10, HTTP 409.	Concurrent edit handling was missing.
Broadcast timing	New FR-RT-10: broadcast after commit; FR-RT-07 rejects non-member subscriptions; FR-RT-08 refetches on reconnect.	Avoids clients seeing rolled-back data.
Positioning	FR-CARD-05/06 and 6.1: BIGINT gap-based positions, midpoint computation, rebalance, same-board check.	Efficient drag-and-drop persistence.
Access errors	New FR-AUTH-12: 404 for non-members.	Avoid leaking resource ids.
CSRF	NFR-SEC-08 reworded: disabled for header-based JWT.	Original wording contradicted stateless JWT design.
Flyway version	Section 7: version managed by Spring Boot; PostgreSQL module note.	'9.x' conflicted with Spring Boot 3 defaults.
Scope	Deferred list added (1.4): attachments, avatar upload, account deletion, email verification, presence, Redis caching.	Keeps first release within about 2 months.
Dashboard	Clarified as cross-board; completion chart uses completed_at.	Consistency with Done definition.
Non-functional	Added NFR-PERF-07 (no N+1), NFR-SEC-12 (rate limiting), NFR-REL-05 (atomic writes), test matrix and concurrency tests in 5.5.	Testable quality requirements.
API	Added GET /cards/{id}/comments, move payload, error codes; demo reset path set to /api/v1/demo/reset.	Completeness and consistency.

A.2 Version 1.1 to 1.2

Area	Change	Reason
Scope split	Added Section 1.5 Scope Priority Split (Must / Should / Defer). References renumbered to 1.6.	Duplicate 1.5 numbering; table of contents was missing the split.
Priorities	Activity log, basic dashboard, profile, overdue indicator and demo core set to Must in 1.5 and 2.2; FR-ACT-06 raised to M; dashboard and demo split into core and extras in 2.2.	1.5, 2.2 and requirement tables disagreed.
Demo reset path	/api/v1/demo/reset used everywhere.	1.5 previously said /api/demo/reset.
Timeline	8-10 weeks (about 2 months) used in 1.5, 2.5 and Appendix A.	1.5 and 2.5 gave different figures.
Token refresh	FR-AUTH-03 note: concurrent 401s share one refresh call.	Rotation would otherwise log users out on parallel requests (AD02).
Soft delete	AD08 clarifies that @SQLRestriction does not cover native or aggregate queries.	Dashboard queries must filter deleted_at explicitly.
Decisions log	Added Appendix B: Architectural Decisions Log (AD01-AD10).	Records rationale for onboarding and interviews.

Area	Change	Reason
Housekeeping	Version set to 1.2, status Final. Appendix B placed before the end-of-document marker.	Version history and document structure were out of date.

A.3 Version 1.2 to 1.3

Area	Change	Reason
AI section	Added Section 10: Future Enhancements (Version 2) — AI Integration. Defines AI feature scope (V2-FR-AI-01 through V2-FR-AI-05), design rules (R1-R6), subscription tier mapping, and API endpoints.	Formalizes the AI roadmap discussed during V1 planning. Keeps V1 scope unchanged while documenting V2 AI direction.
Document conventions	Section 1.2: [Version 2] covers both payment gateway and AI integration.	Consistency with new Section 10.
Product scope	Section 1.4 Out of Scope lists AI-powered features alongside the payment gateway.	Clarity for implementers and evaluators.
Functions summary	Added AI row to Section 2.2.	Completeness.
Glossary	Added LLM and Prompt injection terms.	Supports Section 10 terminology.
V1 requirements	No changes to Sections 3-7. All V1 requirements remain Final as of v1.2.	V1 scope is frozen.

A.4 Version 1.3 to 1.4

Area	Change	Reason
Database	PostgreSQL replaced by MySQL 8.0+ (InnoDB, utf8mb4) in 2.1, 2.4, 2.5, 2.6, 4.3 and 7.	Project database choice.
Table naming	Column entity stored in table board_columns; FR-BOARD-11 and the data model updated.	COLUMN is a reserved word in MySQL.
Data types	Timestamps as DATETIME(6) in UTC, booleans as BOOLEAN, token_hash as CHAR(64), emails lower-cased (6.1 conventions).	MySQL type behaviour: TIMESTAMP 2038 limit, no native boolean, case-insensitive default collation.
Concurrency	Card moves lock the target column row; deadlocks retried. Added FR-CARD-06 note, NFR-REL-06 and AD-13.	InnoDB default REPEATABLE READ differs from PostgreSQL READ COMMITTED; concurrent moves must be serialized.
Migrations and tests	Section 7: Flyway needs the flyway-mysql module; backend tests run against Testcontainers MySQL.	MySQL-specific tooling.
Decisions log	Appendix B renumbered to AD-01..AD-12 (as in v1.3), with AD-01 updated for MySQL and AD-13, AD-14 added.	Record MySQL decisions.
V1 functional scope	No functional requirement was added or removed.	Database change only.

Appendix B: Architectural Decisions Log

This appendix records key architectural decisions made during the development of the SRS.

ID	Decision	Rationale
AD-01	MySQL 8.0+ (InnoDB) as the sole database. No NoSQL.	Relational model fits the hierarchical board/column/card structure. InnoDB provides the transactions and row-level locking that card moves and optimistic locking rely on. JSON columns are available if needed. Simplifies deployment and backups.
AD-02	DB-backed refresh tokens with rotation. Redis made optional.	Enables token revocation on logout and password change. Rotation prevents token replay. Redis adds infrastructure without enabling any V1 requirement.
AD-03	Gap-based BIGINT positioning (gaps of 1000) for columns and cards.	Moving an item requires updating only one row. Midpoint computation avoids cascading updates. Renumbering only when gap < 2 keeps positions stable.
AD-04	Optimistic locking (@Version) on cards.	Cards are the most concurrently edited entity. Optimistic locking avoids the complexity of pessimistic locks and works well with the WebSocket broadcast model (clients converge on the broadcast state).
AD-05	Soft delete (deleted_at) on boards and cards.	Allows recovery of accidentally deleted data. Simplifies audit trails. All queries must include the filter — enforced by convention and tested. Native and aggregate queries bypass @SQLRestriction and must filter explicitly.
AD-06	WebSocket broadcast after DB commit.	Prevents clients from seeing rolled-back data. Each event includes the sender's user id so clients can ignore their own echo.
AD-07	Board-level authorization in the service layer.	Controller checks authentication; service checks board membership and role. Prevents authorization bypass through direct API calls.
AD-08	Single Owner per board, cannot be removed or demoted.	Simplifies the permission model. Ownership transfer is deferred to V2.
AD-09	Viewers may comment on cards.	Resolves the contradiction between 'Viewer is read-only' and 'any board member can collaborate'. Comments are a form of collaboration, not content modification.
AD-10	STOMP over WebSocket (not SSE or polling).	Full-duplex communication. STOMP provides message routing and subscription management. Spring has mature STOMP support.
AD-11	Flyway for all database migrations. No manual SQL.	Version-controlled schema changes. Reproducible deployments. Seed data managed alongside schema.
AD-12	AI features server-side only, env-gated, suggestion-only (V2).	Prevents API key exposure. Allows the feature to be disabled without affecting core functionality. Suggestion-only model preserves all existing write-path controls (optimistic locking, activity log, WebSocket broadcast).
AD-13	Row lock on the target column during card moves; bounded deadlock retry.	InnoDB defaults to REPEATABLE READ. Two concurrent moves in one column could compute the same midpoint or collide during a rebalance. Locking the column row serializes them, and a retry handles rare deadlocks.
AD-14	Table board_columns, DATETIME(6) in UTC, utf8mb4.	COLUMN is a reserved word in MySQL. TIMESTAMP overflows in 2038. utf8mb4 supports full Unicode (e.g., emoji in card titles).

— End of Document —